

The Arche-Cell: A Hardware-Bound Reactive Component for Tripartite Cognitive Architectures

Bob Kowalski

2026

Abstract

This paper presents the Arche-Cell as a local, memoryful, generative primitive designed for integration into larger cognitive architectures. It is explicitly framed as a low-level, high-speed reactive substrate: the hardware-bound actuator of a tripartite AI system (AI-1, AI-2, AI-3). Its function is to provide ultra-low-latency, ultra-low-power control in response to local sensor states and an externally computed valence field. To address the combinatorial constraints of hardware implementation, the standard operating mode restricts the Hamming search radius to $r = 1$ (single-bit flips), reducing the candidate space from tens of thousands to $W + 1$ states per tick. The structural complexity metric is defined as a strictly local function (number of 0-1 transitions in a circular 1D chain), enabling an incremental $O(1)$ update. All terms in the objective functional are explicitly normalised to $[0,1]$ to ensure commensurability. Inter-frame valence dynamics are handled via a precomputable fixed-point decay with explicit error bounds. The formal apparatus of the cell (state space, admissibility correspondence, objective functional, selector operator, and tie-breaking policy) is defined as a direct instantiation of the Arche Selector operator of Kowalski (2026), so that the general theorems of that framework, including the Attractor Basin Partition theorem and the self-loop monotonicity corollary, apply to the Arche-Cell without modification. The paper provides a complete engineering specification, including a fair benchmark comparing the Arche-Cell to a neural network baseline on the same FPGA fabric, ensuring static power overhead is controlled for. The project is offered as a falsifiable engineering hypothesis, with open problems explicitly catalogued.

Keywords: reactive control, hardware acceleration, tripartite architecture, finite-state machine, low-power AI, FPGA, selector operator.

1. Introduction: The Component View

Embodied artificial intelligence requires multiple time scales of computation [1, 2]. High-level planning (pathfinding, mapping, strategic reasoning) operates at millisecond-to-second rates [3]. Low-level control (stabilisation, reflexive avoidance, motor synchronisation) operates at microsecond rates and demands deterministic latency and minimal power [4, 5].

The Arche-Cell is designed for the latter domain. It is a specialised reactive component, not a substitute for deep neural networks. It receives a field of valence, a scalar signal per unit, from an external predictor (AI-2). It produces a stream of low-level actions with bounded latency and constant per-tick cost.

Key departure from standalone formulations: The Arche-Cell is not a general-purpose cognitive primitive. It is a hardware-bound actuator, designed to complement higher-level AI systems [3]. It does not replace planning, learning, or strategic reasoning; it executes their outputs with deterministic speed and efficiency.

Formally, the Arche-Cell is a per-tick, per-cell instantiation of the Arche Selector operator introduced in Kowalski (2026), which defines a multivalued selector on a finite state space independently of interpretation. Section 2 makes this correspondence explicit and carries over the notation S , $\mathcal{L}$, F , $\mathcal{A}$, τ from that paper.

2. Formal Definition

Each Arche-Cell i is a local structure

$$\square_i = \langle s, M_i^\wedge(t), \mathcal{L}, \Pi, \Xi \rangle$$

where:

- $s_i \in S$, $S = \mathbb{B}^W$: the finite state space (Axiom 1 of Kowalski, 2026), with state width $W = 64$, fixed.
- $M_i^\wedge(t) \in \mathbb{B}^{(T \times W)}$: temporal buffer of the last T states ($T = 8$, fixed).
- $\mathcal{L}: S \rightarrow 2^S \setminus \{\emptyset\}$, $\mathcal{L}(s) = \{s' \in S : d_H(s, s') \leq 1\}$: the admissibility correspondence (Axiom 2), restricted to Hamming radius $r = 1$.
- Π : cell index $\rightarrow \mathbb{R}^2$, a fixed spatial projection (2D grid). Π is a hardware layout device external to the Selector axioms; it plays no role in the dynamics of $\mathcal{L}$, F , or $\mathcal{A}$, and only fixes which cells are spatial neighbours for the purposes of $\mathcal{N}(i)$ in Section 2.3.
- $\Xi(s')$: the structural complexity metric (Section 2.1), one of the components of the objective functional F .

Correspondence with the Arche Selector operator. The general framework of Kowalski (2026) is instantiated per cell, per tick, as follows:

Selector symbol	Role in Kowalski (2026)	Arche-Cell instantiation
S	finite state space	$\mathbb{B}^W$, $W = 64$
$\mathcal{L}$	admissibility correspondence	Hamming ball of radius 1 around s_i , including s_i itself
F	objective functional	normalised weighted sum of Ξ , Γ and $\hat{V}$ (Section 2.3)
$\mathcal{A}$	selector operator	$\mathcal{A}^\wedge(t)(s) = \arg \max \text{ over } s' \in \mathcal{L}(s) \text{ of } F^\wedge(t)(s')$
τ	tie-breaking policy (Definition 2)	fixed deterministic rule (e.g. lexicographic on bit index), needed for a complete deterministic system per Property 3

Because F depends on t through the buffer $M_j^\wedge(t)$ and the cached valence $\hat{V}(t)$, the operator used here is a time-indexed instance $\mathcal{A}^\wedge(t)$, a parameterised member of the family $\mathcal{A}_\alpha$ described in Property 4 of Kowalski (2026), with $\alpha \mapsto t$. At each fixed t , $\mathcal{A}^\wedge(t)$ is itself a Selector operator in the exact sense of Definition 1, so every general theorem of that paper (perpetual walk, finite recurrence, invariance, attractor basins) applies to the frozen-time dynamics.

Notational correction. In the present instantiation the symbol τ is reserved, as in Kowalski (2026), for the tie-breaking policy. The source formulation of this paper used τ_{cell} and τ_{decay} for two hardware time constants (tick period and valence decay constant), which collides with that reserved symbol and would make the update rule below ambiguous. To avoid this collision, the two hardware time constants are renamed T_{cell} (tick period) and T_{decay} (valence decay constant) throughout this paper. This is a purely notational correction: no numerical value or derivation changes.

A second overload in the source formulation used the symbol $\square$ both for the cell structure itself and for the spatial neighbourhood function $\square(i)$ in Section 2.3. The cell structure is written $\square$ here (as above) so that $\square(i)$ is left unambiguous as "the set of spatial neighbours of cell i under Π ".

2.1. Structural Complexity Metric Ξ

To guarantee an incremental $O(1)$ update, Ξ is defined as the number of 0-1 transitions in a circular one-dimensional binary chain:

$$\Xi(s') = \sum_{k=1 \text{ to } W} [s'_k \neq s'_{(k \bmod W) + 1}]$$

where $[P]$ is the Iverson bracket (1 if P is true, 0 otherwise), and $(k \bmod W) + 1$ maps $k = W$ to 1, completing the cycle. The explicit parentheses remove any ambiguity regarding operator precedence.

Periodic Boundary Conditions: This definition explicitly imposes $s_{(W+1)} = s_1$, making the chain circular. Flipping a bit at index k affects exactly two transitions: $(k-1, k)$ and $(k, k+1)$, where indices are taken modulo W . This holds for all k , including the boundaries $k = 1$ and $k = W$. The locality proof is thus uniform and without edge-case exceptions.

Hardware Implementation: The circular transition count is computed via:

$$\Xi(s) = \text{popcount}(s \oplus \text{rotr}(s, 1))$$

where rotr is a circular right rotation. This uses a single XOR, a rotation (zero-cost in hardware), and a popcount, ensuring minimal logic depth and $O(1)$ evaluation.

Proof of Locality: Flipping bit k affects only transitions $j = k-1$ and $j = k$ (indices modulo W). Therefore $\Delta \Xi$ depends only on the 3-bit circular neighbourhood $(s_{(k-1)}, s_k, s_{(k+1)})$. This is strictly local.

Maximum Value: For a circular chain, the maximum number of transitions occurs in an alternating pattern (e.g. 101010...). In this case $\Xi_{\max} = W$. For $W = 64$, $\Xi_{\max} = 64$.

2.2. Similarity Metric sim

The similarity between a candidate state s' and a memory buffer M_j is defined as the normalised Hamming similarity over the temporal window:

$$\text{sim}(s', M_j^{\wedge}(t)) = 1 - (1 / (T \cdot W)) \cdot \sum_{k=1 \text{ to } T} d_H(s', s_j^{\wedge}(t-T+k))$$

where $d_H(\cdot, \cdot)$ is the Hamming distance. This yields $\text{sim} \in [0,1]$, with 1 indicating perfect identity with the entire buffer and 0 indicating no overlap.

Normalisation: Since $\text{sim} \in [0,1]$ and Γ is the average of sim over neighbours, $\Gamma \in [0,1]$. Thus $\Gamma_{\max} = 1$.

2.3. Update Rule with Inter-Frame Extrapolation

The objective functional $F^{\wedge}(t): S \rightarrow \mathbb{R}$ that instantiates Axiom 3 of Kowalski (2026) is defined component-wise as:

$$F^{\wedge}(t)(s') = \Xi(s') / \Xi_{\max} + \lambda \cdot \Gamma(s', \in \bigcup_{j \in \mathcal{J}(t)} M_j^{\wedge}(t)) / \Gamma_{\max} + \mu \cdot (1/2) \cdot (\hat{V}(s', \Omega, t) / \hat{V}_{\max} + 1)$$

The selector operator instance at tick t is then, exactly as in Definition 1 of Kowalski (2026):

$$\in \bigwedge_{t \in \mathbb{N}} \left(\bigwedge_{s \in \mathcal{L}_i(s_i^\wedge(t))} F^\wedge(t, s) \rightarrow \max_{s' \in \mathcal{L}_i(s_i^\wedge(t))} F^\wedge(t, s') \right)$$

and the deterministic hardware update rule follows Definition 2 of the same paper:

$$s_i^\wedge(t+1) = \tau \left(\bigwedge_{s \in \mathcal{L}_i(s_i^\wedge(t))} s \right)$$

where τ is a fixed deterministic tie-breaking policy (e.g. lexicographic on bit index). Ties occur with probability zero for generic real-valued sensor input, but τ must still be specified for the system to be a complete deterministic system in the sense of Property 3 of Kowalski (2026).

Remark (self-loop and monotonic ascent). Because $\mathcal{L}_i(s)$ is a Hamming ball of radius 1, it includes s itself as the $k = 0$ term of $C(r, W)$ (Section 5.1). Every cell therefore has a self-loop in the sense of Property 1 of Kowalski (2026), and the hypothesis of Corollary 1 of that paper is satisfied: for fixed t ,

$$F^\wedge(t, s_i^\wedge(t+1)) \geq F^\wedge(t, s_i^\wedge(t))$$

holds at every tick, with equality if and only if $s_i^\wedge(t)$ already maximises $F^\wedge(t)$ over $\mathcal{L}_i(s_i^\wedge(t))$. This gives the Arche-Cell a formal per-tick monotonic-ascent guarantee inherited directly from the general Selector framework rather than assumed separately. Note that this guarantee is intra-tick only: because F itself changes from $F^\wedge(t)$ to $F^\wedge(t+1)$ as the buffer and the cached valence update, global monotonic ascent of a single fixed objective across many ticks is not implied and is not claimed.

Normalisation Consistency: The valence term is explicitly mapped to the interval $[0,1]$ via the transformation $(1/2)(x + 1)$, where $x \in [-1,1]$. This ensures that all three terms of F are strictly commensurable.

Where:

- $\Xi_{\text{max}} = W$ (64 for $W = 64$).
- $\Gamma_{\text{max}} = 1$.
- $\hat{V}_{\text{max}} = \max(\epsilon, \max_t |V(t)|)$, where $\epsilon > 0$ is a small positive constant (e.g. 10^{-9}) to prevent division by zero when the valence field is identically zero during calibration.
- $\hat{V}$ is the externally computed, cached valence field with decay:

$$\hat{V}(t) = V \cdot \exp(-(t - t_{\text{last}}) / T_{\text{decay}})$$

where V is the most recent valence field from AI-2, t_{last} is the timestamp of the most recent update, and T_{decay} is a time constant.

- $\lambda, \mu \in \mathbb{R}^+$ are dimensionless coupling constants.

2.4. Inter-Frame Decay: Hardware-Aware Implementation with Correct Error Bounds

The decay is expressed in real time. For a clock period $T_{\text{cell}} = 10$ ns and a decay constant $T_{\text{decay}} = 5$ ms, the per-tick factor is:

$$\gamma = \exp(-T_{\text{cell}} / T_{\text{decay}}) \approx 0.999998$$

Fixed-Point Approximation: To avoid floating-point multipliers, γ is approximated as $\gamma \approx 1 - 2^{-19}$, where $2^{-19} = 1.9073486328125 \times 10^{-6}$.

Correct Error Calculation: The exact per-tick factor is $\gamma_{\text{exact}} = \exp(-2 \times 10^{-6}) \approx 1 - 2 \times 10^{-6} + 2 \times 10^{-12}$. The difference between the exact and approximate factors is:

$$\Delta\gamma \approx 9.265 \times 10^{-8}$$

The relative error per tick in the factor γ is:

$$\varepsilon = \Delta\gamma / \gamma_{\text{exact}} \approx 9.3 \times 10^{-8}$$

Over a full decay interval of $N = T_{\text{decay}} / T_{\text{cell}} = 500,000$ ticks, the accumulated relative error in the final valence value is bounded by:

$$(1 + \varepsilon)^N - 1 \approx \exp(N \cdot \varepsilon) - 1$$

Substituting $N \cdot \varepsilon \approx 500,000 \times 9.3 \times 10^{-8} = 0.0465$:

$$\exp(0.0465) - 1 \approx 0.0476$$

This corresponds to an accumulated error of approximately 4.8% at the full decay horizon. This is well within acceptable bounds for reactive control applications, where slow valence drift is tolerable.

Implementation: This approximation consumes zero DSP blocks and operates in a single cycle using a subtraction (for $1 - 2^{-19}$) and a multiplication by γ . The multiplication can be implemented as $\hat{V}_{t+1} = \hat{V}_t - \hat{V}_t \gg 19$, which uses only shifts and additions.

3. The Tripartite Integration Architecture

Component	Function	Clock Rate	Substrate
AI-1 (Arche-Cell Grid)	Reactive control, reflex.	100 MHz to 1 GHz	FPGA (LUTs + BRAM)
AI-2 (Predictor)	Generates V from world model.	100 Hz to 1 kHz	CPU / DSP / GPU
AI-3 (Executive)	Strategic planning, goal setting.	1 Hz to 10 Hz	CPU

AI-1 does not plan. It reacts. AI-2 and AI-3 set the field that guides AI-1 [3].

4. Temporal Decoupling: Preserving σ_0

The per-tick cost of the Arche-Cell must remain constant and independent of AI-2 complexity.

- Tick time (T_{cell}): FPGA clock cycle (e.g. 10 ns).
- Field update time (T_{field}): Interval at which AI-2 recomputes V (e.g. 10 ms).
- Cache policy: V is stored in BRAM, written via DMA [5].

Per-tick cost with incremental similarity optimization:

$$\sigma_0 = \kappa \cdot (T \cdot W + (C(r, W) - 1) \cdot T)$$

where $C(r, W) = \sum_{k=0}^r C(W, k)$. The first term $T \cdot W$ is the cost of computing the base Hamming distance for the parent state once per tick. The second term $(C(r, W) - 1) \cdot T$ is the cost of incrementally updating the similarity for each candidate that differs by exactly one bit flip. With $r = 1$, $C(1, 64) = 65$, so the number of single-flip candidates is $C(1, 64) - 1 = 64$. For $W = 64$, $T = 8$:

$$\sigma_0 = \kappa \cdot (512 + 64 \cdot 8) = \kappa \cdot (512 + 512) = \kappa \cdot 1024$$

This replaces the previous estimate of $\kappa \cdot 1032$, reflecting a more precise count that excludes the redundant incremental update for the parent state.

5. Practical Hardware Constraints and Optimizations

5.1. The Combinatorial Wall

For $W = 64$ and $r = 3$, the candidate set size is 43,745 per cell. For $N = 4096$ cells, this yields approximately 1.79×10^8 evaluations per tick, infeasible on mid-range FPGAs [4].

Solution: Set $r = 1$. Candidates = 65 per cell, of which one is the self-successor ($k = 0$ term) and 64 are single-bit flips. Total = 266,240 evaluations per tick.

5.2. Incremental Ξ Update, $O(1)$

Flipping a single bit at index k , the delta depends solely on the circular 3-bit neighbourhood. The $O(1)$ claim holds uniformly for all k , including boundaries.

5.3. Incremental Similarity Update (Performance Optimization)

Computing $\text{sim}(s', M_j)$ from scratch costs $O(T \cdot W)$ per candidate. Since $r = 1$, each non-self candidate differs from the parent state by exactly one bit flip.

Optimization: Compute the total Hamming distance between the parent state $s_i^{(t)}$ and each buffer entry M_j once. For a candidate s' differing by a single bit flip at index k , the Hamming distance to buffer entry M_j changes by either $+1$ or -1 depending on whether the flipped bit matches the corresponding bit in M_j . This reduces the per-candidate cost from $O(T \cdot W)$ to $O(T)$ (checking only the flipped bit against the T historical states). This yields a significant reduction in dynamic power and combinational logic.

5.4. Ξ via Bitwise Popcount with Rotation

The transition count $\Xi(s) = \text{popcount}(s \oplus \text{rotr}(s, 1))$ is implemented directly in hardware using a native popcount instruction or a parallel popcount tree. This reduces logic depth to $O(\log W)$ gates, enabling higher clock frequencies. The rotation is zero-cost in hardware (wire rerouting).

6. Relaxation via Sensor Dynamic Variance (SDV)

The relaxation mechanism ($\theta \rightarrow 0$) is triggered by a formally defined Sensor Dynamic Variance (SDV). This is an engineering heuristic designed to detect environmental turbulence and trigger a temporary reduction in the coherence threshold, allowing the Arche-Cell to remain responsive in highly dynamic conditions.

Definition (SDV) with Dimensional Consistency and Hard Bound:

$$SDV(t) = (1/M) \cdot \sum_{m=1}^M \min(\|sensor_m(t) - sensor_m(t - \Delta t)\| / R_m, 1)$$

where Δt is a short window (e.g. 10 ticks), M is the number of raw sensors, and R_m is the maximum physical range of sensor m , as specified in the hardware datasheet.

Rationale: The $\min(\cdot, 1)$ function guarantees that each term is strictly bounded to $[0,1]$, even if a sensor experiences an unexpected outlier exceeding the calibration range. This preserves the mathematical guarantee that $SDV(t) \in [0,1]$ for all possible sensor inputs, ensuring deterministic behavior in hardware.

Relaxation rule:

- If $SDV(t) > SDV_threshold$: $\theta_{(t+1)} = \max(0, \theta_t - \delta)$
- If $SDV(t) \leq SDV_threshold$ for 10 consecutive ticks: $\theta_{(t+1)} = \min(\theta_0, \theta_t + \delta)$

The threshold $SDV_threshold$ is a free parameter calibrated empirically (see Section 7).

7. Calibration Protocol

Free parameters: $\theta_0, \lambda, \mu, T_decay, SDV_threshold$. $\delta = \theta_0 / 4$.

Cost function:

$$J = \alpha \cdot RMS_error + \beta \cdot relaxation_fraction$$

Calibration procedure:

- Collect dataset from robot/drone logs.
- Simulate the Arche-Cell in software for parameter combinations.
- Evaluate using J .
- Select minimizer and refine via Bayesian optimization.

8. Engineering Specification

Resource / Parameter	Specification
Clock Frequency	100 MHz
Grid Size	$64 \times 64 = 4096$ cells
State Width & Buffer	$W = 64$ bits, $T = 8$ states
Search Radius	$r = 1$ (65 candidates/cell/tick)

Total Evaluations / Tick	266,240 parallel candidate evaluations
BRAM Usage	< 300 KB
FPGA Resource Usage	< 200,000 LUTs (mid-range compatible)
Control Loop Latency	200 ns (20 cycles @ 100 MHz)
Projected Power	To be determined by synthesis
Decay Approximation	Fixed-point shift (zero DSP blocks)
Similarity Update	Incremental $O(T)$ per candidate
Ξ Computation	Bitwise popcount($s \oplus \text{rotr}(s, 1)$)
Ξ_{max}	$W = 64$
σ_0	$\kappa \cdot 1024$ (with incremental similarity)

9. Falsifiable Benchmark

Task: Stabilise a quadrotor drone in a wind tunnel under gust velocity up to 10 m/s.

Comparisons:

- Baseline: PID controller.
- Neural baseline: Quantised 3-layer neural network on the same FPGA fabric.

Success Criteria (against Neural Baseline):

- RMS position error within 15%.
- Total dynamic power $\leq 50\%$ of the neural network's dynamic power [4].
- Max control loop latency $< 10 \mu\text{s}$.

Pre-registration: Full protocol to be registered on OSF.

10. Open Problems

ID	Problem Statement	Status
1	Formal proof of the trilemma with explicit entropy definition.	Open
2	Derivation of SDV_threshold from first principles.	Open (Empirical)
3	Convergence proof under Lipschitz V .	Conjecture
4	Generalisation of incremental updates for $r > 1$.	Open
5	Scaling laws for grid size vs. physical task complexity.	Open
6	Optimal T_{decay} for varying dynamics.	Open
7	FPGA synthesis and power measurement.	Open (Empirical)
8	Formal sufficiency proof of circular Ξ for stabilisation tasks.	Open
9	Characterisation, in the sense of Open Problem 7 of Kowalski (2026), of the	Open

	classes of tie-breaking policies τ that leave the Arche-Cell's basin structure unchanged across the full 4096-cell grid.	
--	---	--

11. Conclusion

The Arche-Cell is a hardware-bound reactive component engineered for tripartite integration. Its formal core, the state space S , the admissibility correspondence $\mathcal{L}$, the objective functional F , the selector operator $\mathcal{A}$, and the tie-breaking policy τ , is a direct per-tick instantiation of the Arche Selector operator of Kowalski (2026). Because each cell's admissibility ball includes a self-loop, Corollary 1 of that paper applies and guarantees monotonic ascent of F within each fixed tick, a property established here formally rather than assumed.

Hardware optimizations, including incremental similarity updates, bitwise popcount with rotation, and fixed-point decay, ensure that the model is implementable on resource-constrained FPGAs with deterministic sub-microsecond latency.

The project is offered as a falsifiable engineering hypothesis, with the remaining open problems explicitly catalogued for future work.

References

- [1] Brooks, R. A. (1991). Intelligence without representation. *Artificial Intelligence*, 47(1-3), 139-159.
- [2] Pfeifer, R., & Bongard, J. (2006). *How the Body Shapes the Way We Think: A New View of Intelligence*. MIT Press.
- [3] Albus, J. S. (1991). A reference model architecture for intelligent system design (RMAISD). *IEEE Control Systems Magazine*, 11(3), 23-30.
- [4] Indiveri, G., & Liu, S. C. (2015). Memory and information processing in neuromorphic systems. *Proceedings of the IEEE*, 103(8), 1379-1397.
- [5] Kuon, I., & Rose, J. (2007). Measuring the gap between FPGAs and ASICs. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 26(2), 203-215.
- [6] Kowalski, B. (2026). The Arche Selector operator: A formal framework for discrete state transitions. SSRN.